

MediScan

Visual Medicine Finder

Implementation Plan

A Multi-Model Deep Learning Web Application

Submitted by: Karthik Unnikrishnan

Institution: Nirmal College of Science and Commerce

Date: June 16, 2026

Project Overview

MediScan is a deep learning-powered Django web application targeting Indian users — particularly the elderly, illiterate, and accessibility-constrained population. Users photograph a medicine strip or a doctor's handwritten prescription. The system automatically reads the medicine name, retrieves cheaper generic alternatives with the same salt composition, displays side effect severity levels, and checks for dangerous drug combinations. No typing is required at any stage.

Final Architecture Decisions

Model	Chosen Method	Reason
Model 1 Strip OCR	YOLOv8s (fine-tuned) + TrOCR-base-printed (pre-trained)	Transfer learning from COCO reduces data requirement. Pre-trained TrOCR handles printed medicine text without fine-tuning.
Model 2 Prescription HTR	TrOCR-base-handwritten (fine-tuned on RxHandBD) + OpenCV segmentation	Consistent TrOCR architecture across both OCR models simplifies Django integration. RxHandBD provides 5,578 gold-standard medical word crops.
Model 3 Generic Finder	CSV lookup + SQLite + rapidfuzz matching	~800K medicine records across 3 CSVs. No neural network needed — pure database lookup with fuzzy brand-name matching.

Model 4 Side Effects + DDI	SQLite lookup (SIDER + DrugBank + Mohneesh)	FDA-grade side effect data (SIDER 4.1) + 191K drug pairs (DrugBank) covers clinical-level drug interaction detection.
Model 5 Anomaly Detection	DISCARDED	Dataset too small (267 images), domain gap too large (lab vs. phone camera). Documented as future scope.

Technology Stack

Backend	Django 5.x, Python 3.11, SQLite
Deep Learning	PyTorch 2.x, HuggingFace Transformers, Ultralytics YOLOv8
Computer Vision	OpenCV, Pillow, Albumentations
Data	pandas, rapidfuzz, SQLAlchemy
Frontend	HTML5, Tailwind CSS (CDN), Vanilla JS
Training Tools	Jupyter Notebook, Roboflow (annotation), Label Studio

Django Project Folder Structure

```
mediscan/ <-- Django project root
mediscan/ <-- Project settings package
settings.py urls.py core/ <-- Main Django app (views, templates)
views.py urls.py models.py ml/ <-- ML inference modules (loaded at startup)
strip_ocr.py <-- Model 1: YOLOv8 + TrOCR pipeline
prescription_htr.py <-- Model 2: OpenCV seg + TrOCR pipeline
generic_finder.py <-- Model 3: fuzzy match + SQLite query
drug_info.py <-- Model 4: side effects + DDI lookup
training/ <-- One-time scripts (run before Django)
augment_yolo.py train_yolo.py train_htr.py build_medicine_db.py
build_drug_db.py saved_models/ <-- Trained weights (.pt files)
db_cache/ <-- medicines.sqlite, drugs.sqlite
media/ <-- User-uploaded images
static/ <-- CSS, JS templates/ <-- HTML templates
home.html result.html manage.py requirements.txt
```

Implementation Phases

Phase 1

Day 1

Environment & Project Scaffold

Create virtual environment

Open terminal in project folder. Run: `python -m venv venv` then activate it (Windows: `venv\Scripts\activate`). All packages install here, isolated from the rest of your system.

Install all dependencies

Run: `pip install django ultralytics transformers torch torchvision opencv-python pillow pandas rapidfuzz albumentations reportlab` Save to requirements.txt with: `pip freeze > requirements.txt`

Create Django project

Run: `django-admin startproject mediscan .` (the dot keeps it in the current folder). Then create apps: `python manage.py startapp core` and `python manage.py startapp ml`

Create folder structure

Manually create: `training/`, `saved_models/`, `db_cache/`, `media/`, `templates/` inside the project root. Add these to `INSTALLED_APPS` in `mediscan/settings.py` along with 'core' and 'ml'.

Configure media & static in settings.py

Add `MEDIA_ROOT = BASE_DIR / 'media'` and `MEDIA_URL = '/media/'` to `settings.py`. Add `STATICFILES_DIRS = [BASE_DIR / 'static']`. Run `python manage.py migrate` to create the initial database.

Verify setup

Run: `python manage.py runserver` — browser should open to Django's default welcome page at `http://127.0.0.1:8000`. If it shows 'The install worked successfully!', Phase 1 is complete.

Checkpoint — Phase 1 complete when Django welcome page loads in browser with zero errors in terminal.

Phase 2

Days 2–5

Model 1 — Strip OCR: Data Prep & Training

Augment YOLO dataset (112 → ~1,400 images)

Write `training/augment_yolo.py` using Albumentations library. Apply per image: random blur, brightness shift $\pm 30\%$, horizontal flip, perspective warp, JPEG compression noise. Output images + matching .txt label files in same YOLOv8 format. Run script and verify augmented count.

Annotate 300-400 mobile-captured images

Upload images from DS1.3 (Mobile-Captured Pharma Packages) to Roboflow free tier at `roboflow.com`. Draw bounding boxes around medicine strips. Export as YOLOv8 format. Download and merge with augmented DS1.1 dataset.

Fine-tune YOLOv8s for medicine strip detection

Write `training/train_yolo.py`: load `yolov8s.pt` (pre-trained on COCO), fine-tune on merged dataset for 50-100 epochs. Save best weights to `saved_models/strip_detector.pt`. Monitor `mAP@50` metric — target above 0.80.

Test TrOCR pre-trained on strip crops

Load `microsoft/trocr-base-printed` from HuggingFace. Crop 20-30 medicine strip regions from DS1.3 images, pass each crop to TrOCR, measure character error rate. If CER below 15%, use pre-trained as-is (no fine-tuning needed). Save model path references.

Write ml/strip_ocr.py inference wrapper

Combine both models: image in → YOLOv8 detects strip region → crop → TrOCR reads text → return medicine name string. Include confidence threshold check: if YOLO confidence below 0.5, return 'strip not detected' error.

Checkpoint — Test full Model 1 pipeline in Jupyter notebook — photo in, medicine name string out — before proceeding.

Phase 3

Days 6–10

Model 2 — Prescription HTR: Fine-tune & Test

Prepare RxHandBD training data

Write training/prepare_rxhandbd.py: read Train_Label.csv (4,463 rows) + Test_Label.csv (1,115 rows). Pair each image path (Train_Set/P*.jpg) with its ground truth word label. Apply handwriting-specific augmentations: elastic distortion, random slant ± 10 degrees, ink dilation/erosion. Target: ~10,000-12,000 samples after augmentation.

Fine-tune TrOCR-handwritten on RxHandBD

Write training/train_htr.py: load microsoft/trocr-base-handwritten, fine-tune using HuggingFace Seq2SeqTrainer for 10-15 epochs with AdamW optimizer. Evaluate on Test_Label.csv after each epoch. Save best checkpoint to saved_models/prescription_htr/. Target character error rate below 20%.

Build OpenCV line segmentation pre-processor

Write the page-splitting logic in ml/prescription_htr.py: (1) Convert image to grayscale, (2) Adaptive threshold to isolate text, (3) Horizontal projection profile to find line gaps, (4) Slice image into individual text lines, (5) Sort slices by y-coordinate (top to bottom), (6) Pass each slice to fine-tuned TrOCR, (7) Join all outputs.

Extract medicine names from raw HTR output

After TrOCR produces raw text (e.g. 'Tab. Amoxicillin 500mg 1-0-1 x 5 days'), use regex patterns to extract medicine names: look for Tab., Cap., Inj., Sy. prefixes or patterns like 'word + Nmg/ml'. Return list of extracted names.

Drop DS2.2 from training pipeline

The 129 unannotated images in Handwritten Medical Prescriptions Collection have no ground-truth labels. Do not include in training. Use 3-5 of these images only for visual demo during viva — show what the model can handle.

Checkpoint — Test full Model 2 pipeline in Jupyter notebook — prescription image in, list of medicine names out — before proceeding.

Phase 4

Days 11–13

Models 3 & 4 — SQLite Database Build

Build medicine database for Generic Finder

Write training/build_medicine_db.py: load all 3 CSVs (A-Z 250K, 1mg 348K, Mohneesh 196K) using pandas. Normalize salt_composition column to lowercase. Deduplicate on (salt + manufacturer) pair. Write to db_cache/medicines.sqlite. Create index on salt_composition. Total process takes 5-10 minutes due to file sizes.

Build drug info database for Side Effects + DDI

Write training/build_drug_db.py: decompress SIDER meddra_all_se.tsv.gz and meddra_freq.tsv.gz into SQLite tables side_effects and se_frequency. Load db_drug_interactions.csv (191K rows) into drug_interactions table with index on (drug1, drug2). Parse Mohneesh drug_interactions JSON column as supplementary Indian brand DDI data.

Write ml/generic_finder.py

Function signature: find_generics(medicine_name) -> list of alternatives. Steps: (1) Fuzzy match medicine_name to known brand names using rapidfuzz (threshold 85), (2) Retrieve salt composition, (3) SQLite query for all medicines with same salt, (4) Sort by price ascending, (5) Return top 5 with name, price, manufacturer.

Write ml/drug_info.py

Two functions: get_side_effects(drug_name) -> top 3 side effects with Common/Uncommon/Rare labels from SIDER meddra_freq. check_interactions(drug1, drug2) -> interaction warning string from DrugBank. Both use SQLite queries with rapidfuzz name matching as fallback.

Checkpoint — Test all four ml/ modules independently in Python shell with real medicine names (e.g. 'Dolo 650', 'Amoxicillin').

Phase 5

Days 14–19

Django Backend — Views, API & Pipeline

Configure Django models and settings

In `core/models.py`: create `ScanResult` model with fields: `image` (`ImageField`), `scan_type` (`CharField`: 'strip' or 'prescription'), `extracted_name` (`TextField`), `created_at` (`DateTimeField` `auto_now_add`). Run `makemigrations` and `migrate`.

Load ML models at startup using AppConfig

In `ml/apps.py`, override `ready()` method to load YOLOv8 weights, TrOCR processor + model, and open SQLite connections. Store as module-level globals in `ml/__init__.py`. This is critical — loading models per request would cause 3-5 second delays on every scan.

Build main ScanView (POST endpoint)

In `core/views.py`: receive uploaded image via POST, save to `media/`, detect `scan_type` from form field, route to Model 1 or Model 2 inference, pass extracted name(s) to Models 3 and 4, collect all results into a dictionary, save `ScanResult` to DB, return `JsonResponse` with full results.

Wire up URL routing

In `core/urls.py`: `path("", views.home, name='home')`, `path('scan/', views.scan, name='scan')`, `path('result/', views.result, name='result')`. Include `core.urls` in `mediscan/urls.py`. Add media URL serving for development.

Handle edge cases and errors

Wrap entire inference pipeline in `try/except`. Return structured error responses: 'Strip not detected in image', 'Medicine name not recognized', 'No generics found in database'. Log errors to Django's default logger. Set file size limit (`MAX_UPLOAD_SIZE = 10MB`) to prevent large file abuse.

Test all API endpoints with Postman or curl

POST a real medicine strip image to `/scan/` with `scan_type=strip`. Verify JSON response contains: `medicine_name`, `generics[]`, `side_effects[]`, `interactions` (if applicable). Fix any inference or DB query failures before moving to frontend.

Checkpoint — Django backend complete when a real photo upload returns correct JSON results in browser developer tools.

Phase 6

Days 20–25

Frontend, Integration & Final Testing

Build home page (templates/home.html)

Two scan mode buttons: 'Scan Medicine Strip' and 'Scan Prescription'. Camera input: — this opens rear camera directly on Android/iOS (no app needed). Image preview using JS `FileReader` API. Submit via `fetch()` to `/scan/` endpoint. Show loading spinner during inference.

Build result page (templates/result.html)

Four sections: (1) Detected medicine name with confidence score, (2) Generic alternatives table with columns: Name, Price (INR), Manufacturer, (3) Side effects list with Common/Uncommon/Rare badges, (4) Drug interaction warning panel (red, shown only when DDI detected). Large fonts and high contrast for elderly user accessibility.

Add Tailwind CSS styling

Use Tailwind via CDN: . No build step needed. Apply clean card layouts, responsive grid, accessible color contrast (WCAG AA minimum). Test on mobile screen size.

End-to-end testing with real photos

Test with minimum: 10 medicine strip photos (including blurry, angled, partial strips), 5 prescription photos (different handwriting styles), 3 cases with 2+ medicines for DDI check. Log: OCR accuracy, generic match rate, page load time. Fix failures.

Prepare demo scenarios for viva

Prepare 4 demo cases: (1) Common medicine — Dolo 650 strip, (2) Drug interaction — Aspirin + Warfarin prescription, (3) Generic savings — branded antibiotic vs cheap alternatives, (4) Elderly use case — blurry phone photo of medicine strip. Run python manage.py runserver and demo from localhost.

Checkpoint — Project complete when all 4 demo scenarios run end-to-end without errors in a live browser session.

Timeline Summary

Phase	Description	Days	Output
1	Environment & project scaffold	Day 1	Django runs, folder structure ready
2	Model 1 — Strip OCR training	Days 2–5	strip_detector.pt, strip_ocr.py
3	Model 2 — Prescription HTR	Days 6–10	prescription_htr/, prescription_htr.py
4	Models 3 & 4 — Database build	Days 11–13	medicines.sqlite, drugs.sqlite
5	Django backend & API	Days 14–19	Working /scan/ JSON endpoint
6	Frontend & final testing	Days 20–25	Live web app, demo ready

Total estimated timeline: 25 days of focused development work

All datasets are already downloaded and organized in the Datasets/ folder. Models 3 and 4 require no training — only database build scripts. Model 5 (Anomaly Detection) has been formally excluded and documented as future scope. This document covers implementation from current state (datasets ready) to a fully functional web application.